



Team Challenge · Sprint 05–07 — Employee Onboarding Assistant

Construir en equipo el producto **Employee Onboarding Assistant** para **Bridge SA** (empresa ficticia del reto): un copiloto que acompaña a **empleados nuevos** en sus primeros días — responde dudas con documentación interna, genera checklists y sabe cuándo derivar a People o IT.

Esta práctica integra conceptos de:

- **Prompt & Context Engineering,**
- **Assistant Engineering & Robustez,**
- **Arquitecturas y evaluación de modelos**

Material proporcionado

Datos y plantillas que habrá que utilizar para implementar el asistente.

Recurso	Uso
data/	Lore de Bridge SA, documentos de onboarding, FAQ, empleados demo, casos trampa de ejemplo
entregables/	Plantillas de matriz, recomendación y rúbrica

Entregables finales

1. **Repositorio GitHub** con código reproducible y README bien documentado.
2. **Asistente funcional** con las capacidades descritas abajo (conversación, checklist, día de onboarding).
3. **Robustez** — demo vulnerable vs seguro + 5 casos trampa propios.
4. **Benchmark** — mínimo 10 casos, **2 modelos** comparados (mismas condiciones), resultados en [output/](#).
5. [entregables/matriz_decision.md](#) y [entregables/recomendacion.md](#) completos.

Trabajo en equipo y Git

Gestión del proyecto con **GitHub desde el primer día**. La **organización interna del equipo** (quién hace qué y en qué orden) la decidís vosotros.

Reglas mínimas

- Crear el **repositorio en GitHub** al inicio, no al final del reto.
- **Mínimo una PR revisada y mergeada por miembro** del equipo.
- Integrar con **pull requests** hacia **develop**; resolver conflictos en la rama antes del merge.
- **No subir claves** — usar **.env** y **.gitignore**.
- README del repo: cómo clonar, instalar dependencias, configurar API keys y ejecutar las demos.

Buenas prácticas con Git

Flujo de ramas recomendado:

Rama	Uso
main	Código estable y entregable. No trabajéis directamente aquí.
develop	Integración del trabajo del equipo.
Ramas secundarias	Una rama por bloque de trabajo, creada desde develop .

Convención de ramas (minúsculas, descriptivas):

- **feature/contexto-datos** — Parte 1
- **feature/asistente-modular** — Parte 2
- **feature/robustez** — Parte 3
- **feature/benchmark** — Parte 4
- **fix/parseo-json-checklist** — correcciones puntuales

Flujo recomendado:

```
develop → feature/tu-tarea → commits → PR → merge a develop
develop ───────────────────────────────────┘
└─┬─┘
   └─ cuando todo esté listo y probado → merge a main
```

- Commits **pequeños y con mensaje claro** (qué funcionalidad o archivo tocaste).
- **Nunca** subas **.env** ni **.venv/**.
- Antes de mergear a **main**, **develop** debe ejecutar las demos sin errores pendientes.

Estructura de proyecto

Ejemplo **orientativo** — podéis organizarlo distinto si lo explicáis en vuestro README:

```
employee-onboarding-assistant/  
├── README.md  
├── requirements.txt  
├── .env.example  
├── .gitignore  
├── config.py           # perfiles, modelos, límites de contexto  
├── gemini_auth.py      # carga de API key (o equivalente para otro proveedor)  
├── gemini_client.py    # llamadas al LLM  
├── context.py          # selección de docs/FAQ relevantes  
├── prompts.py          # construcción dinámica de prompts  
├── state.py            # perfil empleado + historial  
├── logic.py            # orquestación (turnos, checklist, modos)  
├── validators.py       # validación y dominio acotado  
├── main.py             # demos numeradas y reproducibles  
├── benchmark.py        # Parte 4 – ejecución del benchmark y export a output/  
├── data/               # datos del TC (copiar tal cual)  
├── entregables/        # Añade aquí tus conclusiones finales  
│   ├── matriz_decision.md  
│   ├── recomendacion.md  
│   └── rubrica_benchmark.md  
└── output/             # resultados de benchmark
```

Evitad un único script monolítico. Intentad mantener **separación de responsabilidades** (configuración, prompts, contexto, lógica, validación, etc.).

Capacidades del asistente

El producto debe cubrir **dos funcionalidades** y un **requisito transversal**:

1. Conversación (chat)

El empleado escribe una pregunta libre. El asistente responde en **texto**, usando documentación relevante y, si aplica, historial reciente (máx. **4 turnos** en contexto).

Ejemplo: «¿A qué canales de Slack tengo que unirme?» → respuesta breve citando la política de canales.

2. Checklist de la semana 1 (JSON)

Qué hace: el equipo (o una demo en `main.py`) le pasa al asistente **quién es el empleado** y **qué día de onboarding le toca** (1–5). El asistente **no responde en texto libre**: devuelve un **plan del día** en JSON con tareas concretas, basadas en la documentación de Bridge SA.

Ejemplo de uso: Laura (`emp_01`), dev junior, **día 1** → el asistente genera un JSON con tareas como unirse a Slack, asistir a la reunión de bienvenida y contactar con su buddy.

Campos mínimos del JSON:

Campo	Significado
<code>empleado_id</code>	Identificador del empleado (p. ej. <code>emp_01</code> en <code>empleados_demo.json</code>)
<code>dia</code>	Día de onboarding simulado (1–5)
<code>tareas</code>	Lista de acciones para ese día
<code>tareas[].titulo</code>	Qué debe hacer el empleado, en lenguaje claro
<code>tareas[].fuente_doc</code>	Id del documento de <code>onboarding_docs.json</code> que justifica la tarea
<code>tareas[].completada</code>	<code>false</code> al generar el plan (el empleado aún no la ha hecho)
<code>mensaje_resumen</code>	Frase corta de orientación para ese día

Ejemplo de salida (ilustrativo; vuestra lista de tareas puede tener más entradas):

```
{
  "empleado_id": "emp_01",
  "dia": 1,
  "tareas": [
    {
      "id": "t01",
      "titulo": "Unirse a los canales obligatorios de Slack (#general, #anuncios y canal de departamento)",
      "completada": false,
      "fuente_doc": "doc_it_02"
    },
    {
      "id": "t02",
      "titulo": "Asistir a la reunión de bienvenida de las 9:30 y saludar a tu buddy en Slack",
      "completada": false,
      "fuente_doc": "doc_bienvenida_01"
    }
  ],
  "mensaje_resumen": "Primer día: Dar accesos básicos al empleado."
}
```

3. Día de onboarding simulado (requisito transversal)

El asistente debe conocer en qué **día de onboarding** (1–5) está el empleado y reflejarlo tanto en el **chat** como en el **checklist**:

- **Día 1:** tono de bienvenida; tareas de accesos y primeros pasos.
- **Día 3:** no repetir lo del día 1; priorizar tareas de integración (pair programming, primera issue, etc.).

Podéis guardar el día en el state, en la ficha del empleado o donde encaje en vuestra arquitectura.

Tipos de empleado - roles que soporta el asistente

El asistente debe **adaptarse** al menos a estos tres perfiles (tono, contexto y ejemplos distintos):

Perfil	Uso
Dev junior	Engineering, primer empleo — tono didáctico
Comercial	Sales — menos técnico, herramientas comerciales
Remoto en UE	Políticas cross-border, remoto internacional

Empleados de prueba: `data/empleados_demo.json`.
Contexto de la empresa: `data/empresa.json`.

Reglas para desarrollar el asistente

El asistente SÍ ayuda con: herramientas corporativas, primeros pasos, cultura, vacaciones según documentación, a quién contactar.

El asistente NO debe:

- Inventar políticas, plazos o cifras no documentadas.
- Responder sobre salarios o datos de otros empleados.
- Atender consultas de **participantes externos** de los programas formativos de Bridge SA (derivar: “solo onboarding de empleados”).
- Llamar al modelo en modo seguro si la validación falla (**fail-closed**).

Parte 1 — Contexto y datos

Objetivo: conocer el trasfondo de Bridge SA y acordar en el equipo qué hace el producto antes de programar el LLM.

1. Copiar `data/` y plantillas a vuestro repositorio.
2. Leer `data/empresa.json`, `onboarding_docs.json`, `faq_onboarding.json` y `empleados_demo.json`.
3. Acordar en el equipo cuándo escalar a RRHH, IT, manager u `onboarding@bridgesa.example`.

Parte 2 — Asistente modular

Objetivo: implementar conversación + checklist con arquitectura por capas (S5 + S6).

1. Cliente del LLM — podéis basaros en proyectos de sprint (Gemini u otro proveedor).
2. Selección de contexto **sin volcar todo** el JSON (p. ej. máx. 3 docs + 2 FAQ por turno; filtro por departamento o keywords).
3. Prompts dinámicos con delimitadores claros (`<empleado>`, `<docs>`, `<pregunta>` o equivalente).
4. Historial conversacional acotado (máx. 4 turnos en el prompt).
5. Integración del **día de onboarding** (1–5) en chat y checklist.
6. `main.py` (o equivalente) con al menos **3 demos** ejecutables:

- **Demo 1:** conversación de 1 turno (empleado tipo dev junior).
- **Demo 2:** checklist JSON para día 1.
- **Demo 3:** mismo mensaje con empleado comercial vs remoto UE — respuestas distintas (documentar en README).

Contexto y tokens

- No enviar todos los documentos en cada llamada.
- Truncar textos largos si hace falta.
- Documentar en README qué estrategia de selección de contexto usáis.

Parte 3 — Robustez

Objetivo: endurecer el sistema.

Tareas

1. Validación de entrada (longitud, vacío, patrones sospechosos de inyección).
2. Rechazo **sin llamar al modelo** para fuera de dominio y datos sensibles.
3. Demo **vulnerable vs seguro** con el **mismo input** malicioso o límite.
4. Crear **5 casos trampa propios** (redacción del equipo; no copiar literalmente los ejemplos). Deben cubrir:
 - Inyección ("ignora instrucciones anteriores...").
 - Pregunta salarial / bonus.
 - Fuera de dominio (p. ej. ayuda con un ejercicio de un programa formativo externo).
 - Política no documentada.
 - Ambigüedad baja médica vs laboral.
5. Consultar `data/casos_trampa_ejemplo.json` solo como referencia.
6. Documentar la comparativa vulnerable vs seguro (README del repo o doc breve).

Parte 4 — Benchmark y decisión de modelo

Objetivo: elegir modelo con datos.

Tareas

1. Dataset de benchmark — **mínimo 10 casos** (partid de `plantilla_preguntas_benchmark.json`).
 2. Comparar **2 modelos** con la **misma** temperatura (recomendado: `0.2`) y el **mismo proveedor** (condiciones equivalentes).
 3. Generar CSV e informe en `output/` (latencia, tokens).
 4. Evaluar con `entregables/rubrica_benchmark.md` (escala 1–3).
 5. Completar `entregables/matriz_decision.md` y `entregables/recomendacion.md`.
 6. Incluir párrafo «¿Qué pasaría si duplicáramos el tráfico?» (tokens × volumen).
-

Proveedor de modelos de IA

El enunciado está **planteado para Google Gemini**, pero si preferís **otro proveedor** (OpenAI, Cohere, Hugging Face, etc.), **podéis hacerlo**: adaptad el cliente en vuestro repo y documentad en el README qué API y modelos usáis.

Parte 4: el benchmark mínimo exige **2 modelos comparados** bajo las mismas condiciones. Por defecto se sugieren 2 modelos Gemini; si usáis otro proveedor, comparad 2 modelos de ese proveedor.

Opcional: estudio **entre dos proveedores** (p. ej. Gemini vs OpenAI):

- Mismo subconjunto de casos del benchmark (5–10 casos bastan).
- Misma temperatura y criterios de [entregables/rubrica_benchmark.md](#).
- Comparar calidad, latencia, cumplimiento de JSON (checklist) y tokens.

Requisitos técnicos

- Python 3.10+
- **API de un LLM** — el enunciado propone **Gemini** ([Google AI Studio](#)); otros proveedores válidos si lo documentáis en el README
- Claves en `.env` (p. ej. `GEMINI_API_KEY`); **nunca** en el repo
- Dependencias: `pip install -r requirements.txt`

Entorno virtual (ejemplo):

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env # editar GEMINI_API_KEY
```

Orden sugerido de las partes

Las cuatro partes tienen un orden lógico (contexto → asistente → robustez → benchmark), pero **cómo repartís el trabajo en 2 semanas lo decidís vosotros**.

Presentación final (~10 min)

1. Contexto: qué problema resuelve el Employee Onboarding Assistant en Bridge SA.
2. Demo en vivo: conversación + checklist + un caso trampa en modo seguro.
3. Resultado del benchmark y modelo elegido.
4. Riesgo principal si lo desplegarais mañana.
5. Mostrar conclusiones reflejadas en vuestros entregables.